

UNIVERSAL DATA SUITE

Design and Development of Packet-Decoding and Data-Simulation
Applications for Network Telemetry Analysis

(Rohit Yadav, B.Tech. 7CSE7Y, A2305223472)

Amity University Noida

ABSTRACT

This report presents the Universal Data Suite, which I built as a single qmake project written in C++11 against Qt 5.10 and compiled for 32-bit Windows with the mingw toolchain. From that one project, and a single shared code core compiled into each, two independent desktop applications are produced. The first, UniversalWiresharkLogReader, is a reader that decodes UDP and TCP telemetry, taken either from a replayed .pcap or .pcapng capture file or from a live network stream, and turns the raw bytes into scaled engineering values that it exports to Excel and JSON. The second, UniversalDataSimulator, is a transmitter that sends user-defined hexadecimal and NMEA-0183 messages over UDP, TCP, or a serial COM port, so that the reader and the simulator serve as each other's natural test rig: one emits the traffic that the other receives and decodes. A substantial part of my early effort was a deliberate learning phase, during which I worked through the Qt framework, the qmake and mingw build chain, packet capture and the pcap and pcapng file formats, the UDP and TCP transports, the structure of Interface Control Documents, and the byte-level layout of protocol messages before committing to a design.

The technical heart of the project is a shared data model paired with an encode/decode contract that makes the simulator the exact byte-for-byte inverse of the reader. A FieldDefinition records a name, a one-based byte offset, a length, a data type, and a scaling resolution; a MessageDefinition groups such fields and adds an optional header, comparison options, a data format, and its connection bindings. The model is deliberately a superset, so each application reads only the parts it needs and ignores the rest. The contract fixes the wire representation precisely: the raw value is the source value divided by the resolution, rounded and written big-endian across the field's length, with two's complement for signed integers, a fixed four bytes for IEEE float and eight for double, and NUL-padded UTF-8 for strings; per-field little-endian is an opt-in that reverses only the numeric and float bytes at the final wire step, while strings and booleans are left as they are and the internal bit editor stays big-endian. Integer values may also be entered in hexadecimal with a 0x prefix. Because the simulator's encoder is capped at eight bytes (a 64-bit quint64), I made the reader's decoder the asymmetric partner: it decodes fields of any length through exact base-256 to base-10 conversion and handles two's complement for wide signed types. Around this contract I implemented a full NMEA-0183 sentence stack terminated by an XOR checksum, and an editable Interface Control Document importer that reads message tables straight from Word .docx files.

Beyond the core contract I carried out the wider engineering that makes the suite usable in practice. A multi-connection transport model lets messages bind by identifier to named UDP, TCP, or serial endpoints, so the reader spins up one receiver per connection and decodes each datagram only against the messages bound to the endpoint it arrived on, while the simulator can fan a single message out to several destinations at once, producing one frame and one history line for each. I wrote a PcapWriter that synthesises Ethernet, IPv4, and UDP or TCP frames to export the simulator's sent history as a pcapng capture, and a headless functional test harness that drives the real import, codec, encode-decode, and pcap code paths rather than mocks; it passes all thirty-three checks and proves a capture round trip in which a frame written by PcapWriter re-parses through the reader's own file reader and packet parser with its

payload, IP addresses, and ports intact. Building the suite gave me concrete, tested experience in Qt desktop development on a fixed 5.10 target, low-level network and serial programming, binary protocol design against an exact encode/decode contract, capture-file and document parsing, and the disciplined version control needed to evolve two applications from one shared core.

CHAPTER 1

INTRODUCTION

1.1 Background

Networked instruments, avionics buses and telemetry links rarely speak in human-readable text; they emit tightly packed binary messages in which every byte position stands for a specific engineering quantity. Once such traffic is captured with a tool like Wireshark and written to a .pcap or .pcapng file, all that survives on disk is a wall of raw hexadecimal payloads that means nothing on its own. Recovering an actual value demands the full recipe for each field: its exact byte offset, its length, whether it is a signed or unsigned integer, a floating-point number or a string, and the scaling resolution that turns the stored integer back into a physical unit. Performing this reconstruction by hand, byte after byte, for every message and every field is slow, mentally taxing and remarkably error-prone, because a single off-by-one in an offset, a forgotten two's-complement sign extension, or the wrong endianness quietly yields a number that looks plausible yet is simply wrong.

The hand-worked approach is also almost impossible to reproduce reliably. A calculation figured out on paper or in a spreadsheet for one capture must be rebuilt from nothing for the next, and the interpretation logic survives only in the analyst's head instead of in a reusable, inspectable definition. A second, mirror-image difficulty surfaced as soon as I asked how such a decoder would even be tested: exercising a receiver normally requires the real source hardware transmitting real messages, which may be costly, absent from a student's desk, or awkward to coax into a specific edge case. Two things are therefore missing at once. First, a way to capture a message's field layout once as a formal definition and then decode any matching payload automatically. Second, a controllable transmitter that can synthesise exactly those messages on demand, so that the decoder and anything downstream can be driven without the original equipment. Answering that paired need is what set this project in motion.

1.2 Project Overview

The Universal Data Suite is a single qmake project, written in C++11 against Qt 5.10 for 32-bit Windows, that builds two separate desktop applications from one shared code core. The first, whose executable is UniversalWiresharkLogReader, is a reader: it decodes UDP and TCP data, either from a .pcap or .pcapng capture file or from a live network stream, into engineering values and exports the results to Excel (.xlsx) and JSON. The second, whose executable is UniversalDataSimulator, is a transmitter: it sends user-defined hexadecimal and NMEA-0183 messages over UDP, TCP or a serial COM port. Rather than duplicate the field-and-message model, the codecs and the numeric logic across two independent programs, I gathered all of that common logic into a shared core that is pulled into both applications through a single qmake .pri include, making it one source of truth compiled twice into two distinct executables.

The two applications are deliberately built around a mutual-inverse relationship, and this is the technical heart of the whole project. The simulator's encoding of a value is defined to be the exact inverse of the reader's decoding of the same bytes: where the reader reads raw bytes and multiplies by a resolution to obtain an engineering value, the simulator takes that

engineering value, divides by the resolution, rounds, and writes the result big-endian across the field's byte length, using two's complement for signed types and fixed IEEE layouts for float (4 bytes) and double (8 bytes). Because both sides are derived from the same shared `FieldDefinition` and `MessageDefinition` data model, a message described once can be transmitted by the simulator and decoded by the reader with byte-for-byte agreement. That turns the correctness of the pair into something I can state as a contract rather than hope for as a coincidence.

A direct consequence of this design is that each application is the other's natural test rig. The simulator can be aimed at the reader's live mode over loopback or a LAN link, so that frames it encodes are received and decoded in real time, letting me confirm that a value entered on the sending side reappears unchanged on the receiving side. Conversely, the reader serves as an independent check that the simulator is producing well-formed frames, since any discrepancy in offset, length, endianness or scaling surfaces at once as a wrong decoded value. This reciprocity meant that throughout development I always had a controllable way to generate known inputs and an independent way to verify the outputs, with no need for the real telemetry hardware the suite is ultimately meant to stand in for.

1.3 Objectives of the Project

The project set out to deliver a working, testable pair of applications resting on a single shared foundation, and to do so under a strict, self-imposed technical discipline. The concrete objectives I pursued are as follows.

- To build a reader that decodes UDP and TCP messages into engineering values from both saved .pcap/.pcapng capture files and a live network stream, including exact decoding of fields wider than eight bytes through base-256 to base-10 conversion.
- To build a simulator that encodes user-defined values into binary frames and transmits them over UDP, TCP or a serial COM port, and that additionally composes and sends NMEA-0183 sentences with correct two-hex-digit XOR checksums.
- To define one shared data model (`FieldDefinition` and `MessageDefinition`) that is a superset of both applications' needs, so that a message described once serves the reader and the simulator alike, each ignoring the fields it does not use.
- To specify and honour an exact encode/decode contract in which the simulator's encoding is the precise inverse of the reader's decoding, covering resolution scaling, two's-complement signing, fixed IEEE float and double layouts, NUL-padded UTF-8 strings and opt-in per-field little-endian byte order.
- To support importing interface control documents from Microsoft Word (.docx) files, auto-detecting the name, offset, length, type and resolution columns along with the offset base, and presenting an editable review before message definitions are built, under lenient rules that fill sensible defaults instead of discarding recoverable fields.
- To provide lossless JSON interchange of the field-and-message model together with Excel (.xlsx) read/write of field lists in both applications, so that a definition authored in one application always loads in the other.
- To implement a multi-connection routing model in which named UDP/TCP/serial endpoints bind to messages by id, letting the reader run one receiver per connection

without mixing traffic and letting a single simulator message fan out to several destinations at once.

- To back the encode/decode contract with an automated, headless functional test harness that exercises the real code paths and proves a pcap round-trip, all while staying strictly within Qt 5.10 and C++11 and applying the reason-plus-solution error philosophy throughout.

1.4 Scope of the Project

This project spans both learning and delivery. As an academic undertaking, it let me work through binary protocol handling, network programming, desktop GUI construction and cross-application data interchange inside one coherent codebase rather than as disconnected exercises. As a deliverable, it produces two genuinely usable Windows applications sharing a common core, developed in stages: I began with the core data model and the encode/decode logic, then added the file-based reader, the network transports and the simulator, and layered on the higher-level features, ICD import, JSON and Excel interchange, the multi-connection model, pcapng writing with a packet inspector, standard output folders with autosave, and the light and dark themes with in-app manuals. Each stage was kept buildable and testable, so the round-trip relationship between the two applications could be re-checked as the suite grew.

The project also draws its boundaries on purpose. On the encoding side the simulator caps a field at eight bytes because it constructs values through a 64-bit integer (quint64) path, whereas the reader deliberately reaches further and can decode fields of arbitrary width; this asymmetry is intentional, reflecting that transmitting very wide fields was out of scope while faithfully decoding them was not. Interface control document import is confined to Microsoft Word .docx files, since those expose a table structure the extractor can walk through QZipReader, and PDF import was weighed but left out because Qt 5.10 offers no PDF module and flat PDF carries no reliable table structure to parse. In the same spirit the suite standardises on JSON and Excel for interchange and drops the earlier CSV support, and it commits to Qt 5.10-compatible APIs with a single vendored non-Qt dependency, the MIT-licensed QXlsx, for Excel; anything demanding a newer Qt or further third-party libraries falls outside the scope I set for this work.

CHAPTER 2

LITERATURE REVIEW

The Universal Data Suite sits at the intersection of several established standards and a strand of academic work, and my reading was deliberately practical: I studied each source for the byte layouts, file conventions, sentence grammars and framework behaviours that I would have to reproduce exactly for the reader's decoder and the simulator's encoder to agree. The material falls into four groups. The transport and internetworking RFCs together with IEEE 802.3 fix how I frame Ethernet, IPv4, UDP and TCP bytes in PcapWriter; the capture-format lineage of libpcap, the pcapng draft and Wireshark governs how I write and independently verify pcapng files; the NMEA-0183 standard supplies the sentence-and-checksum rule that the simulator builds and the reader parses; and the Qt 5.10 reference with ECMA-376 pin the implementation down to a specific framework release and to the .docx and .xlsx packaging I unzip and generate. A final pair of protocol-reverse-engineering papers justified my central abstraction of a message as an ordered list of typed fields. For each source below I record its objective, its key findings, and the precise decision it drove in the reader's decode path or the simulator's inverse encode path.

Table 2.1 Literature reviewed, with the specific design decision each source informed.

S.N o.	Reference	Objective	Key Findings	Relevance to the Project
1	J. Postel, RFC 768, "User Datagram Protocol", 1980	To define a minimal, connectionless datagram service over IP carrying source and destination ports, a length and a checksum.	UDP adds only an eight-byte header and delivers datagrams without ordering, acknowledgement or retransmission, leaving reliability to the application above it.	UDP is the primary transport in both applications: I synthesise its eight-byte header inside PcapWriter, and in the reader's live mode I run one receiver per connection so that each datagram is decoded only against the messages bound to the connection it arrived on.
2	J. Postel, RFC 793, "Transmission Control Protocol", 1981	To specify a reliable, connection-oriented byte stream with sequence numbers, acknowledgements and a three-way handshake.	TCP guarantees ordered, error-checked delivery over a negotiated connection, and separates an active client role from a passive listening server.	The suite offers TCP as an alternative transport: a ConnectionDefinition carries the TCP role and host, the simulator opens a stream sender, and the reader accepts one stream per connection, mirroring the active-client and passive-server distinction the standard defines.
3	J. Postel, RFC 791, "Internet Protocol", 1981	To define addressing and best-effort routing of datagrams between hosts using a variable-length header.	IPv4 fixes the 32-bit address format, the header layout, the protocol field and the header checksum that encapsulate the transport payload.	PcapWriter wraps each UDP or TCP payload in a well-formed IPv4 header, so the synthesised frames re-parse in my own reader and open cleanly in Wireshark with the correct source and destination addresses.

4	IEEE, "IEEE 802.3: Ethernet", IEEE Standards Association	To standardise the Ethernet frame layout and the link-layer framing of network traffic.	The frame prepends destination and source MAC addresses and an EtherType that names the upper-layer protocol carried in the payload.	PcapWriter prepends an Ethernet header to the IPv4 packet, so every synthesised capture is a complete link-layer frame that a standard analyser can dissect end to end.
5	National Marine Electronics Association, "NMEA 0183 Interface Standard"	To define an ASCII sentence protocol for the interchange of navigation and instrument data.	A sentence opens with a talker and formatter, carries comma-separated fields, and closes with an XOR checksum over the body followed by CRLF.	The simulator assembles NMEA-0183 sentences to exactly this grammar - talker, formatter, comma-joined tokens, two-hex-digit XOR checksum and CRLF - and the reader decodes them, with the talker, sentence type and per-field mapping held in the shared MessageDefinition.
6	V. Jacobson, C. Leres, S. McCanne, "libpcap", The Tcpdump Group	To provide a portable library and file convention for capturing and storing network packets.	libpcap established the de facto capture model of timestamped raw frames that downstream tools read back for offline analysis.	My reader ingests capture files in this lineage, and its capture-then-replay workflow is the exact model my offline decode path follows.
7	G. Combs et al., "Wireshark Network Protocol Analyzer", Wireshark Foundation	To provide an interactive tool that dissects captured traffic into human-readable protocol layers.	Wireshark demonstrates layered dissection from the link frame through the transport header to the payload, with a per-packet detail view.	Wireshark is my external ground truth for every pcapng file I write, and its per-packet detail view is the direct inspiration for the simulator's double-click packet inspector.
8	M. Tuexen, F. Risso, J. Bongertz, G. Harris, M. Richardson, "PCAP Next Generation (pcapng) Capture File Format", IETF Internet-Draft	To specify a block-structured, extensible successor to the classic pcap capture file.	pcapng organises a capture into typed blocks, among them a section header, an interface description and enhanced packet blocks with timestamps.	PcapWriter emits these blocks directly, so the simulator can export its sent history as a valid pcapng capture that both Wireshark and my own reader parse.
9	The Qt Company, "Qt 5.10 Reference Documentation"	To document the C++ classes and modules of the Qt 5.10 application framework.	Qt 5.10 supplies cross-cutting facilities for sockets, serial ports, widgets and archive reading, with API behaviour specific to that release.	The entire suite targets Qt 5.10 only, so I depended on this reference for the exact 5.10 behaviour of QUdpSocket, QSerialPort and the gui-private QZipReader I use to open .docx interface documents.
10	ECMA International,	To standardise the	A document is a Zip	The shared ICD extractor unzips

	"ECMA-376: Office Open XML File Formats"	Office Open XML packaging and markup used by .docx and .xlsx files.	package whose parts hold XML describing paragraphs, tables and spreadsheet cells.	a Word .docx and walks its table XML, while the QXlsx-based codec reads and writes field lists as .xlsx - both resting on this packaging standard.
11	J. Caballero, H. Yin, Z. Liang, D. Song, "Polyglot: Automatic Extraction of Protocol Message Format Using Dynamic Binary Analysis", ACM CCS, 2007	To recover the field structure of protocol messages automatically from program execution.	Message formats can be inferred as sequences of typed fields with explicit lengths and boundaries, treating a message as structured rather than opaque bytes.	This reinforced the central abstraction of my design - a message as a sequence of typed fields with explicit lengths and boundaries - which is exactly what my FieldDefinition and MessageDefinition encode.
12	J. Narayan, S. K. Shukla, T. C. Clancy, "A Survey of Automatic Protocol Reverse Engineering Tools", ACM Computing Surveys, 2015	To survey techniques and tools for recovering undocumented protocol formats and semantics.	Field type, offset, length and byte order are the recurring dimensions a format description must pin down, and tolerance for imperfect input matters in practice.	It validated both my lenient ICD import and the explicit per-field endianness and resolution in the data model, and framed my goal as reconstructing a message format from a written specification rather than from binaries.

Summary

Read together, these sources are less a background survey than a specification I had to honour on both sides of the wire. RFC 768, RFC 793 and RFC 791 with IEEE 802.3 fix the exact Ethernet, IPv4, UDP and TCP byte layouts that PcapWriter emits and that the reader interprets when it routes each datagram against the messages bound to the connection it arrived on; the pcapng draft, the libpcap capture model and Wireshark give me a portable file format and an external oracle against which every frame I generate is checked. NMEA-0183 supplies the ASCII talker-formatter-checksum grammar that the simulator assembles and the reader disassembles, while the Qt 5.10 reference and ECMA-376 anchor the implementation - from QUdpSocket and QSerialPort to unzipping a Word ICD with QZipReader and exchanging field lists as Excel through QXlsx. Finally, the reverse-engineering literature confirmed the abstraction on which the whole suite turns: a message as an ordered list of typed fields, each with an offset, a length, a byte order and a scaling resolution, and it justified the lenient import path that lets a user rebuild a format from a specification rather than from binaries. Because the simulator's encode is engineered to be the exact inverse of the reader's decode, it is this shared, standards-grounded model that allows the two applications to serve as each other's faithful test rig.

CHAPTER 3

METHODOLOGY AND SYSTEM ARCHITECTURE

3.1 Development Approach

I built the Universal Data Suite iteratively and incrementally rather than attempting to specify the whole system up front. My starting point was the single fact that defines the project: the simulator's encoder must be the exact byte-for-byte inverse of the reader's decoder. I therefore treated that round trip as the spine of the work, implementing the narrowest possible slice first — an unsigned big-endian integer scaled by a resolution — and only then widening outward. Each new capability, whether a data type, an endianness option, an NMEA sentence, or a transport, was added one at a time and immediately exercised through the encode-then-decode loop, so that at no point did I carry more than one unverified change. This kept the two applications in lockstep even though they compile to separate executables, because a value transmitted by the simulator and recovered intact by the reader is the only proof that the shared contract still holds.

My testing was driven by that same round trip rather than by isolated unit assertions. For every data type I would encode a known engineering value in the simulator's payload builder, capture the resulting wire bytes, feed those bytes back through the reader's extraction engine, and confirm that the recovered value matched the original to within the field's resolution. Signed two's-complement widths, IEEE float and double, NUL-padded UTF-8 strings, and per-field little-endian reversal were each admitted into the code base only after this loop closed cleanly for them. Because the reader can decode fields wider than the simulator's eight-byte encode cap, I verified those wide signed and unsigned cases separately using exact base-256 to base-10 conversion, checking the decimal result against a hand-computed expectation instead of against another piece of my own code.

A design principle I applied uniformly is that every user-facing failure must report both a reason and a solution, never a bare error code. Validation failures are gathered across an entire operation and surfaced together in one dialog, so the user sees the complete picture at once instead of dismissing message after message. I also parse every number in the C locale rather than the machine's regional locale, so that a decimal separator is always a point and offsets, resolutions and hexadecimal values are interpreted identically regardless of the Windows region the tool happens to run under. These two choices — actionable errors and locale-independent parsing — follow from my view that a data tool used in an engineering setting must fail predictably and legibly, because a number silently misread is far more dangerous than an operation that refuses to proceed.

3.2 Development Environment and Tools

I developed the suite on a 32-bit Windows toolchain built around Qt 5.10 and the C++11 standard, with qmake as the sole build system and Wireshark serving as the external reference against which I validated the pcap files I produced. The table below lists each technology I relied on and the specific role it played in the project.

Table 3.1 Development tools and technologies used

S.No.	Technology	Purpose
1	C++11	Implementation language for both applications and the shared core, restricted to language features available under the mingw 32-bit compiler.
2	Qt 5.10	Application framework supplying the widgets, UDP/TCP networking, serial-port access and container/string types used throughout the suite.
3	qmake	Build system; one project builds both executables from a shared .pri include, using source globbing so files are picked up by pattern rather than being listed by hand.
4	mingw (32-bit)	GCC-based compiler and linker toolchain that produces the two Windows executables.
5	QXlsx	Vendored MIT-licensed library for reading and writing Excel .xlsx field lists and result exports; the only non-Qt dependency, linked into both apps.
6	QZipReader	Qt gui-private component used to open Word .docx ICD files as ZIP archives so their tables can be extracted.
7	Wireshark	External packet analyser used to confirm that the pcapng files written by my PcapWriter are well formed and readable by third-party tools.
8	Git and GitHub	Version control and remote hosting of the repository across development machines.
9	Visual Studio Code	Primary editor used for browsing, editing and building the code base.

A constraint I imposed on myself throughout was to stay strictly within Qt 5.10 and avoid any API introduced after that release, so the suite builds cleanly on the fixed toolchain with no conditional compilation. One qmake behaviour repeatedly caught me out: the project files enumerate their sources with a glob, and qmake only re-scans that glob when a .pro or .pri file itself changes. As a result, after adding a new source or header I had to re-run qmake (or delete the build directory) before the file was compiled at all — otherwise it was silently omitted from the build with no error. I record this here because it is exactly the kind of environment-specific trap that costs an afternoon if it is not written down.

3.3 System Architecture

The suite is organised as one qmake project that yields two independent desktop applications from a common body of code. The reader, UniversalWiresharkLogReader, is the receiving and decoding side: it opens a .pcap or .pcapng capture file, or listens on a live UDP or TCP stream, and turns raw bytes into engineering values that it can export to Excel or JSON. The simulator, UniversalDataSimulator, is the transmitting side: it encodes user-defined hexadecimal and NMEA-0183 messages and sends them over UDP, TCP, or a serial COM port. The two are deliberately complementary, so the simulator becomes the natural test rig for the reader's live mode — one program emits the exact frames the other is designed to recover.

Both applications sit on top of a single shared core that holds everything the two sides must agree on: the field and message data model, the JSON and Excel interchange codecs, the ICD extractor, the connection model, the NMEA sentence machinery, the pcap writer, and the

theming and help infrastructure. Around that core each application layers only what is genuinely specific to its role — the simulator its payload builder and data senders, the reader its extraction engine, pcap file reader and packet parsers — while the encode/decode contract embedded in the core keeps the two halves consistent. The overall arrangement is shown in the figure below.

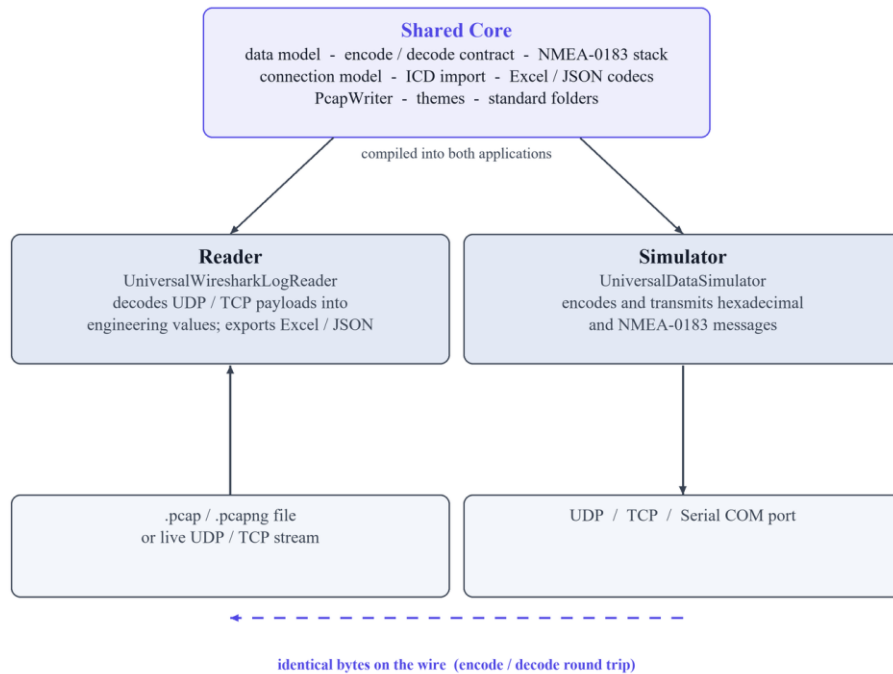


Fig 3.1 Overall architecture of the Universal Data Suite

An important architectural decision is that the shared core is compiled directly into each executable rather than distributed as a static or dynamic library. The common .pri include is referenced by both application project files, so its sources are compiled twice, once into each binary, from a single source of truth. I chose this over a static library deliberately: a Qt static library that carries generated moc output, .ui-derived headers and compiled resource data is fragile to link correctly under the mingw toolchain, and compiling the core into each app sidesteps that fragility entirely at the modest cost of a second compilation. The result is two fully self-contained executables that nonetheless can never drift apart in their understanding of the data model, because they are built from exactly the same core files.

3.4 The Shared Data Model

At the centre of the shared core are two structures that between them describe any message the suite can handle. A `FieldDefinition` names a single field and records its one-based byte offset within the message (stored internally as a zero-based offset equal to the offset minus one), its length in bytes, its data type — signed or unsigned integer, float, double, string, or boolean — and a scaling resolution that relates the raw integer to an engineering value. Alongside these interpretation attributes the same structure also carries the reader's decode configuration and the simulator's send value together with a per-field endianness flag, so that one field description serves both directions of the contract.

A MessageDefinition groups an ordered set of fields and adds the attributes that apply to the message as a whole: an optional header used to recognise the message, comparison options, a data format, an NMEA talker and sentence type, a send frequency in hertz for the simulator, and the identifiers of the connection or connections the message is bound to. Together these two structures are enough to describe both a hexadecimal binary layout and an NMEA-0183 sentence within a single model, which is precisely what lets one ICD import or one JSON file drive both applications.

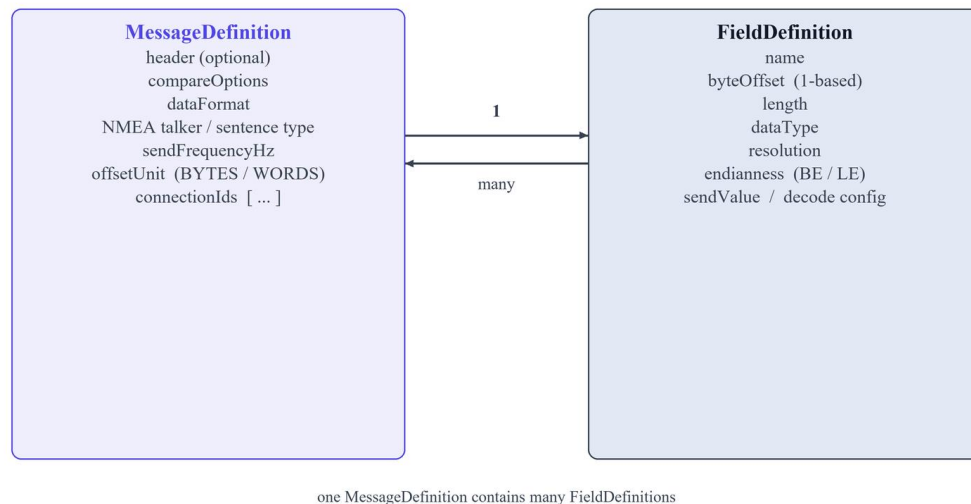


Fig 3.2 The shared message and field data model

The model is intentionally a superset of what either application needs on its own, and each app simply ignores the members that do not concern it: the reader disregards the simulator's send value and send frequency, while the simulator disregards the reader's decode configuration. This union approach means a message defined or imported in one program loads faithfully in the other with no lossy conversion step in between. One member that both share but interpret purely for display is the offset unit: a message may present its field offsets in bytes (the default) or in words, where one word is two bytes, but the stored offset is always kept in bytes — so the words option changes only what the user reads on screen and never the underlying byte arithmetic.

3.5 The Encode/Decode Contract

The encode/decode contract is the heart of the project, and I specified it precisely so that the two applications could be developed against a written rule rather than against each other's incidental behaviour. To encode a value, the simulator computes the raw integer as the value divided by the field's resolution and rounded to the nearest integer, then writes that integer big-endian in exactly the field's declared number of bytes. Signed values are represented in two's complement at a width of length times eight bits, a float is written as a fixed four IEEE bytes, a double as a fixed eight IEEE bytes, and a string as UTF-8 padded with NUL bytes to the field length. Decoding in the reader applies the identical rules in reverse, reconstructing

the raw integer from the bytes and multiplying by the resolution to recover the engineering value.

Because the two sides follow the same rules in opposite directions, they form matched pipelines whose composition is the identity within a field's resolution. The one asymmetry I allowed is deliberate: the simulator's encode is capped at eight bytes because it assembles each field in a 64-bit integer, whereas the reader carries no such limit and can decode a field of any length by converting the raw bytes from base 256 to base 10 exactly, handling two's complement for wide signed types. This lets the reader cope with real capture files that contain fields wider than anything the simulator itself would generate, without ever sacrificing the exactness of the recovered value.

Keeping the contract explicit also made the wide-field and hexadecimal cases tractable, since every stage of the pipeline could be reasoned about as a pure transformation of bytes rather than as UI behaviour. The inverse relationship between the two pipelines is illustrated in the figure below.

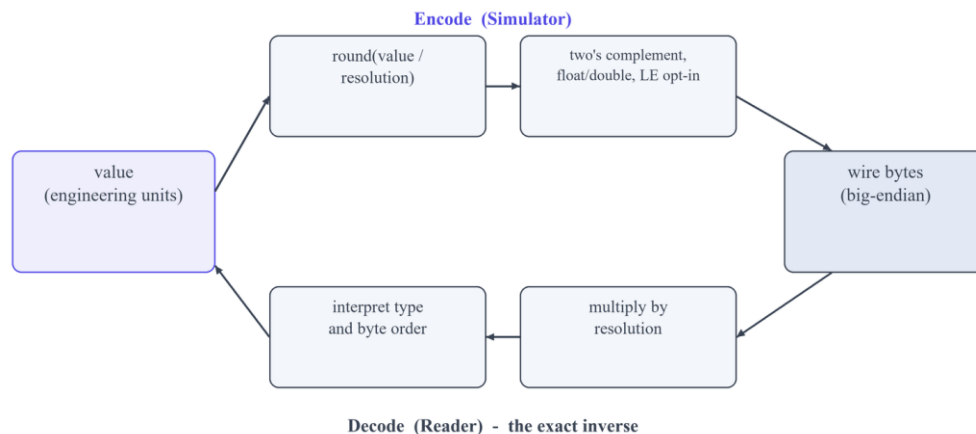


Fig 3.3 Encoding and decoding as inverse pipelines

Two refinements sit on top of the core rule. Per-field little-endian is an opt-in that reverses only the numeric or floating-point bytes at the final wire step, leaving strings and booleans untouched and keeping the internal bit editor big-endian, so a user can match a little-endian device without the rest of the model changing behaviour. NMEA sentences are assembled by a separate but equally strict rule: a dollar sign, the talker, the formatter, the comma-joined data tokens, an asterisk, a two-hexadecimal-digit XOR checksum computed over the sentence body, and a terminating carriage-return line-feed. Finally, integer values may be entered directly in hexadecimal with a 0x prefix — for signed types the hexadecimal is taken as the raw two's-complement bit pattern for the field width — and a read-only Hex column shows the exact bytes as they are typed, so the user watches the wire representation take shape in real time.

CHAPTER 4

IMPLEMENTATION

4.1 Implementation Overview

I organised the whole suite as a single qmake project whose top-level .pro file uses the subdirs template to produce two independent executables: UniversalWiresharkLogReader for the reader and UniversalDataSimulator for the simulator. Rather than copy the common code between them by hand, I gathered every shared class into one directory and surfaced it through a single qmake include file, a shared .pri, that each application's own .pro file pulls in. A .pri is textually spliced into whichever project includes it, so the shared data model, the JSON and Excel codecs, the connection model, the NMEA stack, the ICD extraction layer, the theming and the PcapWriter are all compiled straight into each executable instead of being linked out of a pre-built archive. I chose this compile-the-core-twice arrangement over a shared static library on purpose: a Qt static library that has to drag along its own moc output, generated UI headers and compiled resources is awkward and brittle to link against, whereas recompiling a modestly sized shared core costs almost nothing next to the hours that linker fragility can burn.

Inside that structure the shared core supplies the project's common vocabulary, above all the FieldDefinition and MessageDefinition types alongside the codecs and the connection endpoints, while each application keeps its own main window and its own transport code in a separate folder. I made the data model a deliberate superset of what either program requires: it holds the reader's decode configuration and the simulator's send values and per-field endianness side by side, and each application simply overlooks the members it does not consume. The payoff is that one JSON or Excel file describes a message in full no matter which tool authored it, and encode and decode both read from an identical set of field descriptions. Across the codebase I confined myself to Qt 5.10-compatible C++11 APIs, parsed every number in the C locale so a regional decimal separator can never be misread, and arranged for every user-facing failure to state both its cause and a concrete remedy, gathered into a single dialog rather than a burst of separate pop-ups.

4.2 The Reader Application

The reader, UniversalWiresharkLogReader, is the consuming half of the suite: its task is to turn raw bytes, whether they arrive from a stored capture file or from a live socket, back into the engineering quantities the message was assembled to carry. It operates entirely from the shared MessageDefinition list, so the very descriptions that drive the simulator's transmission also govern the reader's interpretation. In the subsections that follow I set out the decode pipeline that turns bytes into values, the two input modes that feed that pipeline, and the extra handling I built for fields wider than sixty-four bits.

4.2.1 The Decode Pipeline

For each field the pipeline starts by locating its bytes within the payload. Because the stored byteOffset is one-based in the interface, I convert it internally to a zero-based index by subtracting one, then read exactly the declared number of bytes from that position. The interpretation of those bytes turns on the field's dataType and its endianness flag: bytes are

read big-endian by default, but when a field is marked little-endian I reverse only its numeric and floating-point bytes before interpretation and leave strings and booleans untouched, which is the precise mirror of the rule the simulator obeys when it writes them. For signed integer types I read the assembled magnitude as a two's-complement value at length times eight bits, so that the most significant bit correctly signals a negative quantity and the stored pattern maps cleanly back to the original signed number.

With the integer magnitude recovered, I apply the field's scaling resolution by multiplying the raw value by that resolution to obtain the real engineering value, which is the exact inverse of the round-and-divide step the transmitter performed. Floating-point fields skip the scaling path and are decoded as fixed-width IEEE quantities, four bytes for a float and eight for a double. String fields are decoded as UTF-8 and cut off at the NUL padding the encoder inserted, so a name stored in a fixed-length field returns without its trailing filler. Boolean fields are reported as a plain true or false drawn from their byte. Every decoded value is placed into the output row for its message, and it is those completed rows that the reader exports to Excel through QXlsx or to JSON.

4.2.2 File Mode and Live Mode

In file mode the reader opens a .pcap or .pcapng capture and walks it packet by packet through its own PcapFileReader and UdpPacketParser, lifting out each transport payload. It then routes every payload against the configured messages by matching on the destination port, on the payload length, and, where a message declares one, on an optional header signature at the front of the payload. A payload that satisfies a message's port, length and header is decoded against that message's fields; this routing is exactly what lets one capture that interleaves several distinct message types be sorted correctly without the user having to split the traffic apart beforehand.

In live mode the routing is organised around connections rather than a lone socket. Every transport setting lives in a Configure Connections dialog, and starting a live capture requires at least one connection to be defined; on start the reader spins up one receiver per connection instead of a single shared listener. Each datagram is therefore decoded only against the messages bound to the connection on which it actually arrived, with an unbound message treated as matching any connection. This per-connection isolation guarantees that traffic on two different endpoints, say two UDP ports, or a UDP link and a TCP link, is never accidentally cross-decoded, and a message meant for one endpoint can never be populated from bytes that landed on another.

4.2.3 Decoding Wide Fields

The simulator caps its encoding at eight bytes because it assembles each value in a sixty-four-bit unsigned integer, yet a real ICD occasionally defines counters or identifiers wider than that, so I made the reader capable of decoding a field of any length. Instead of packing the bytes into a fixed integer type, the reader carries out an exact base-256 to base-10 conversion, treating each byte as a digit in base 256 and accumulating the decimal result with arbitrary-precision string arithmetic, so no precision is lost however many bytes the field spans. For a wide signed field I first inspect the most significant bit to detect a negative value and then apply two's complement across the full width before the base conversion, so a wide

signed quantity is reported with the correct sign and magnitude. This makes the receive side strictly more capable than the transmit side, which is the right asymmetry: decoding captures produced by other systems demands a tolerance that generating one's own traffic simply does not.

4.3 The Simulator Application

The simulator, UniversalDataSimulator, is the producing half of the suite: it takes the user's field values and renders them into wire bytes that it transmits over UDP, TCP, or a serial COM port. Because it reads the same field and message descriptions as the reader, everything it emits is by construction something the reader already knows how to decode, and that is precisely what makes the two programs each other's natural test rig. The subsections below describe the payload builder that performs the encoding, the hexadecimal entry path and the bit editor behind it, and the construction of NMEA-0183 sentences.

4.3.1 The Payload Builder

I wrote the payload builder to be the exact inverse of the reader's decode pipeline, field for field, so that any value entered in the simulator survives a full round trip through transmission and decoding unchanged. For a numeric field it first derives the raw integer as the value divided by the field's resolution, rounded to the nearest whole number, and then writes that integer big-endian into the field's declared length in bytes. Signed values are written in two's complement at length times eight bits, so a negative quantity produces exactly the bit pattern the reader expects to read back as negative. When a field is marked little-endian, the builder reverses only its numeric or floating-point bytes at the final wire step, precisely as the reader reverses them on the way in, while the internal representation stays big-endian throughout.

Floating-point fields sidestep the resolution scaling and are emitted as fixed-width IEEE values, four bytes for a float and eight for a double, matching the reader's fixed-width float decode. String fields are encoded as UTF-8 and padded with NUL bytes out to the field's declared length, so a short name still fills its full allotment on the wire and decodes cleanly. Once every field has been rendered, the builder concatenates them in offset order into the message payload, which it then hands to the appropriate DataSender for the connection or, when a message is bound to several connections at once, transmits to each of them in turn, producing one frame and one history line per destination.

4.3.2 Hexadecimal Entry and the Bit Editor

To make it straightforward to reproduce an exact byte pattern taken from a specification, I let the user enter any integer field's value in hexadecimal by prefixing it with 0x, and this works for both unsigned and signed integer types. For a signed type the hexadecimal figure is interpreted as the raw two's-complement bit pattern for the field's width rather than as a magnitude, so a value copied directly out of an ICD's hex column reaches the wire byte-for-byte. Beside each field the dialog presents a read-only Hex column that refreshes as the value is typed, showing the exact bytes the field will contribute so the user can confirm the encoding before anything leaves the machine.

Underneath this entry sits a bit editor that always works big-endian internally, whatever the field's declared endianness. Keeping the editor big-endian means the bit and byte positions shown to the user line up directly with the specification's ordering, and the little-endian reversal is applied exactly once, at the very last step when the field's bytes are laid on the wire. That keeps the mental model simple, the specification still reads left to right, and confines the byte-order concern to a single well-defined transformation instead of letting it leak through the editing logic.

4.3.3 NMEA-0183 Sentence Construction

For NMEA-0183 traffic the simulator builds a text sentence rather than a binary payload. It opens with the '\$' start delimiter, appends the two-character talker identifier followed by the sentence formatter, and then adds the field values as tokens joined by commas. It computes the checksum as the XOR of every character between the '\$' and the '*', renders that checksum as two hexadecimal digits after a '*' terminator, and closes the sentence with a carriage-return and line-feed pair as the standard prescribes. Because the reader carries a matching NMEA decoding stack, a sentence the simulator emits parses back into the same talker, formatter and field tokens on the receiving side, so NMEA messages round-trip through the suite just as the binary messages do.

4.4 The Multi-Connection Transport Model

So that a single project could describe traffic flowing over more than one network endpoint, I represented each transport channel as a named `ConnectionDefinition` instead of hard-wiring transport settings into the main window. A connection records its transport kind (UDP, TCP or SERIAL), the local adapter, the port, the TCP role and host, and the serial line parameters, and it carries a stable id. A `MessageDefinition` binds to one or more connections by holding those ids, which cleanly separates a message from the wire it rides on and lets either be reconfigured without disturbing the other. One model then serves both applications: to the reader a connection is a place to receive from, and to the simulator it is a place to transmit to.

On the reader I concentrated every transport setting inside one `Configure Connections` dialog, and I made `Start Live` decline to begin until at least one connection exists, stating the reason and steering the user back to that dialog. When capture begins, the reader launches one receiver per configured connection rather than a single shared socket. Each datagram is decoded only against the messages bound to the connection it arrived on, while any message left unbound is treated as matching every connection. That routing rule is precisely what stops traffic on two different endpoints from being cross-decoded — a real hazard the moment one project can listen on several ports at once.

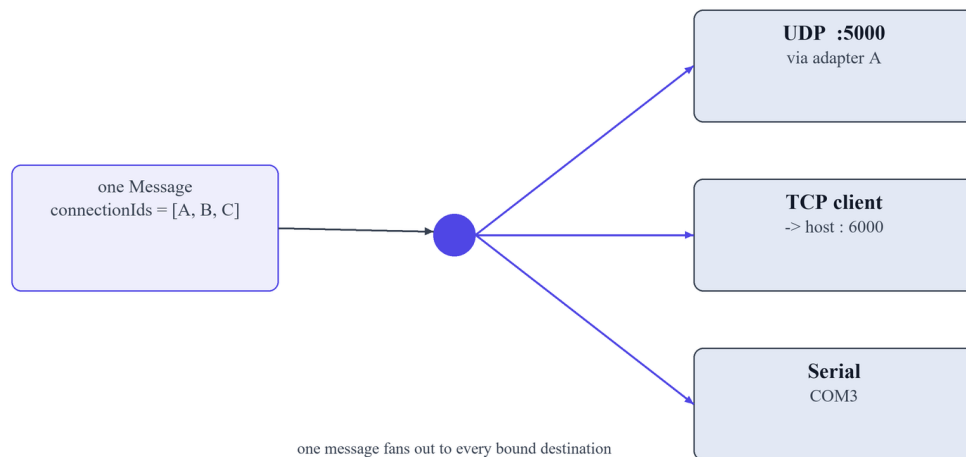


Fig 4.1 Multi-connection routing: one message to many destinations

The simulator applies the same model in reverse for sending, and here a quirk of multi-homed machines forced a deliberate fix. A message can fan out to several connections through a multi-select Connection column; on Start the simulator opens exactly one sender per referenced connection and transmits to every bound destination, emitting one frame and one history line for each. For UDP I added a Send via adapter combo that binds the sending socket to a chosen local interface. Without that bind, a host with more than one network card would quietly route everything to loopback and the packets would never leave for the intended LAN. Binding to the selected adapter address makes the outgoing interface explicit and repeatable, which matters when the reader is listening on a particular card of the same machine or of another host on the network.

4.5 Importing Definitions from Word ICD Documents

In my target domain, Interface Control Documents almost always arrive as Word .docx files, so I built a shared extractor that turns such a document straight into message and field definitions. Since a .docx is really a ZIP archive of XML parts, I open it with QZipReader from Qt's gui-private module, read the document body, and walk its table structures into a neutral in-memory form. Over that I present a table picker, because a genuine ICD interleaves definition tables with revision histories, glossaries and prose, and the user must say exactly which tables describe messages. For each selected table the importer auto-detects the columns holding the field name, byte offset, length, data type and scaling resolution, and it infers the offset base so that a document numbering its bytes from zero or from one is interpreted correctly either way.

I chose not to trust that detection blindly, so the import terminates in an editable review where every extracted field can be corrected before a single MessageDefinition is built. The parsing rules are intentionally forgiving, because ICDs are written by people and are routinely incomplete: a field is dropped only when at least two of the three identifying keys — name, offset and type — are missing, and when just one is absent I fill a sensible default and record

a note stating what I assumed. Duplicate field names inside a message, which crop up whenever a document reuses a label such as `reserved`, are disambiguated with a `_2`, `_3` and onward suffix rather than rejected outright. The upshot is that a messy but usable document imports in one pass, accompanied by a clear account of every assumption, instead of aborting on the first irregular row.

4.6 Excel and JSON Interchange

Because the reader and the simulator share one data model, I wanted definitions to travel freely between them and to be archivable outside either application, so I wrote a JSON codec that serialises the full field and message model without loss. It captures everything the model holds — names, offsets, lengths, data types, resolutions, headers, comparison options, per-field endianness, send values, NMEA settings, the display offset unit and the connection bindings — so a project exported from one application is reconstructed exactly when imported into the other. Alongside JSON I added an Excel codec that reads and writes field lists as `.xlsx` workbooks through the vendored `QXlsx` library, since a spreadsheet is the form engineers actually trade when they hand-edit a field table. Both interchange paths are surfaced symmetrically, as matched import and export actions in each application.

Interchange only earns its keep if it is tolerant, so import is deliberately lenient, in the same spirit as the ICD reader: a file written by one application always loads in the other, and small gaps such as a blank field name are repaired with a generated placeholder and a warning rather than a hard failure. This matters because the two apps exercise overlapping but not identical subsets of the model, and neither should choke on a field the other populated. In the course of this work I removed CSV support entirely; an earlier iteration carried a CSV path, but its lack of typing and its quoting ambiguities made round-trips lossy, whereas Excel preserves cell types and JSON preserves the whole structure. Narrowing the formats to Excel and JSON left me two well-behaved paths in place of three, the third having been a steady source of edge cases.

4.7 Capture Export and Packet Inspection

To make the simulator a credible test rig, its output had to be inspectable with the same tools an engineer would point at real traffic, so I wrote a `PcapWriter` that synthesises complete packets instead of logging bare payloads. For each recorded message it wraps the wire bytes in an Ethernet frame, an IPv4 header and a UDP or TCP header, computing the fields a real network stack would produce, and writes them into a `pcapng` file. The simulator can export its whole sent history as one such capture, turning a session of transmissions into a standard packet file. Because the frames are built to specification, these files open directly in Wireshark, so the simulator's behaviour can be audited with an independent, trusted tool.

For fast inspection without leaving the application, I also made any row in the sent history double-clickable, opening a Wireshark-style packet inspector that shows the layered protocol breakdown and the raw bytes of that single frame. This closes the loop with the reader, which decodes exactly the kind of capture the writer emits, and it let me check the two halves against one another. I proved the round-trip end to end: a UDP frame written by `PcapWriter` re-parses through the reader's own `PcapFileReader` and `UdpPacketParser` with the payload, the source and destination IP addresses and the ports all preserved exactly. Because the identical

bytes satisfy both my parser and Wireshark, the capture format is demonstrably correct rather than merely self-consistent.

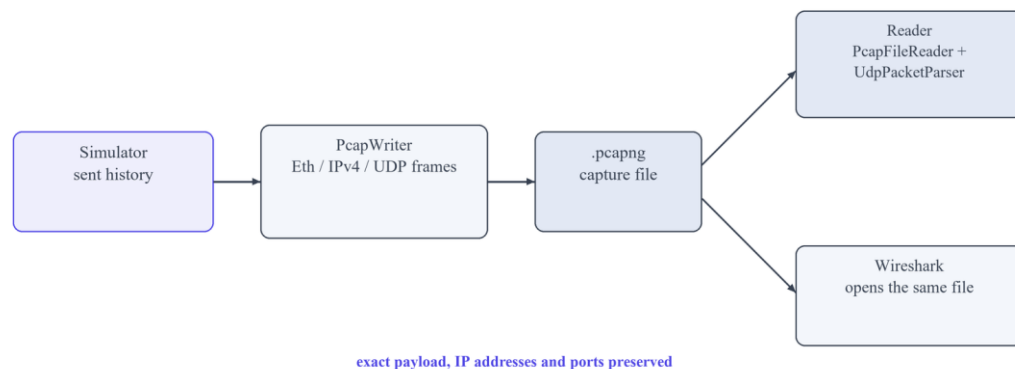


Fig 4.2 The capture round-trip between the simulator and the reader

4.8 Visual Design and Usability

Both applications share a small visual and usability layer that I treated as engineering work in its own right rather than a finishing touch. There are two themes — a Modern Light default and a Slate Dark alternative — switched instantly with Ctrl+T, so the tools stay legible whether the surrounding workspace is bright or dim. Everything the user creates lands under a predictable Documents/UniversalDataSuite tree, with an Output Files folder as the default target for every export and a Projects folder for saved and automatically saved projects. To guard work against a crash or an accidental close, each application autosaves to the Projects folder on a thirty-second timer and restores the last session on the next launch.

For guidance I embedded full HTML user manuals inside each application, opened from the Help menu or the F1 key, with in-page search and a table of contents so a user never has to leave the tool to learn it. I also enforced a firm rule for failures: every user-facing error reports both a reason and a concrete solution, and related errors are gathered into a single dialog instead of a cascade of pop-ups. Numeric input is parsed in the C locale throughout, so a value typed on a machine with regional number formatting is never silently misread — a detail that carries real weight, since a misparsed offset or resolution would corrupt an entire decode.

4.9 Testing and Results

I verified the project at two levels: a headless functional harness that drives the real code paths, and manual confirmation that both applications build and launch cleanly. The harness is itself a console qmake application that pulls in the shared core together with each app's data-path code — the payload builder, the extraction engine, the bitfield and conditional decoders, the pcap file reader and the UDP packet parser — and exercises the genuine import, codec, encode-decode and pcap logic rather than mock stand-ins. It writes its outcomes to a JSON results file, and in the current state all thirty-three of its checks pass. Building the whole suite with a clean qmake and mingw32-make -j4 produces both executables with zero errors, the reader at roughly 2.5 MB and the simulator at roughly 1.9 MB, and both start without crashing.

The single most important thing the tests confirm is the encode/decode contract that is the heart of the project. Because the simulator's encode is defined as the exact inverse of the reader's decode, I could pit them directly against each other: a value encoded to wire bytes and decoded back must return the original, across signed and unsigned integers, floats, doubles and strings, and across both byte orders. The table below summarises the aspects I checked, the method used for each, and the result obtained.

Table 4.1 Summary of testing and validation

Aspect	Method	Result
Clean build of both executables	qmake plus mingw32-make -j4 from a clean tree	Zero errors; reader ~2.5 MB, simulator ~1.9 MB, both launch
Encode/decode round trip	Encode a value through the simulator path, decode it through the reader path, compare to the original	Values recovered exactly for integer, float, double and string types
Wide-field decode	Decode fields longer than eight bytes via exact base-256 to base-10 conversion, including wide signed values	Correct magnitudes and two's-complement signs beyond the 64-bit (8-byte) encode cap
Big-endian and little-endian byte order	Encode and decode the same field in both orders, confirming only numeric/float bytes reverse	Byte order applied correctly; strings and booleans left unchanged
NMEA round trip	Build a sentence from talker, formatter, comma-joined tokens and an XOR checksum, then re-parse it	Sentence structure and two-hex-digit checksum matched on the return path
ICD import lenience	Import tables carrying missing keys and duplicate names through the shared extractor	One-key-missing fields kept with noted defaults; two-key-missing skipped; duplicates suffixed _2, _3
Excel/JSON interchange	Export the model from one app and import it into the other through both codecs	Full model round-trips losslessly; lenient import accepts cross-app files
pcapng capture vs Wireshark	Export sent history as pcapng and open it in Wireshark	Ethernet/IPv4/UDP frames parse correctly in an independent tool
Automated headless harness	Console qmake app driving the real data paths and writing results to JSON	33 of 33 checks pass

Taken together, these results give me confidence that the suite behaves as specified and, more tellingly, that its two applications stay mutually consistent. The pcap round-trip validates the capture format against both my own reader and Wireshark, the encode-decode tests validate the numeric contract in both directions, and the ICD and interchange tests show that real, imperfect input is absorbed without data loss. Every check runs against the actual production code paths rather than reimplementations, so a passing harness is genuine evidence and not a parallel model that could drift away from the shipped behaviour. The verification still outstanding, and noted in the project as pending, is the broader manual end-to-end exercise of the live reader-and-simulator link over real UDP, TCP and serial hardware, which builds directly on the automated foundation established here.

CHAPTER 5

CONCLUSION

5.1 Summary

Across this project I designed and implemented the Universal Data Suite: a single qmake project, written in C++11 and confined to Qt 5.10, that compiles one shared source core into two cooperating Windows desktop applications. The first, UniversalWiresharkLogReader, ingests UDP and TCP traffic from a stored .pcap or .pcapng capture or from a live network stream, decodes the raw payload bytes back into engineering values, and exports the decoded rows to Excel (.xlsx) and JSON. The second, UniversalDataSimulator, transmits user-defined hexadecimal and NMEA-0183 messages over UDP, TCP, or a serial COM port. Because the simulator emits precisely the byte streams the reader is built to interpret, the pair served as each other's test rig for the whole of development, letting me drive the send path against the receive path with no external signal generator or additional hardware in the loop.

What binds the two programs is a single shared data model and a strict inverse relationship enforced between them. One FieldDefinition and MessageDefinition pair, compiled into both executables through a common qmake .pri include, describes every field's name, one-based byte offset, length, data type, scaling resolution, endianness, and connection binding as a deliberate superset, from which each application quietly ignores the members it does not use. Layered on that model is the encode/decode contract that forms the core of the work: the simulator computes $\text{raw} = \text{round}(\text{value} / \text{resolution})$ and writes it big-endian across the field's length, two's complement for signed integers, fixed IEEE widths of four and eight bytes for float and double, and an XOR-checksummed NMEA sentence, while the reader executes the exact arithmetic inverse of each of those steps. Holding the model as one source of truth compiled twice, rather than duplicating the structures or hiding them behind a static library whose moc and resource handling I distrusted, meant every change to the wire format reached the sender and the receiver in the same edit, which is what lets the round-trip between the two applications be provably lossless.

5.2 Learning Outcomes

Carrying the suite from an empty repository to two working executables taught me a great deal that a lecture course cannot, because each design choice had to survive contact with real bytes moving over a real socket. The work sharpened concrete technical skills in systems and network programming and, alongside them, a set of working habits around byte-level debugging, documentation, and disciplined version control. I have grouped the outcomes below into technical and professional strands.

Technical Learning Outcomes

- I became fluent in Windows desktop application development in C++11 on the Qt 5.10 widget framework, targeting the mingw 32-bit toolchain, and covering signal-slot wiring, model-driven tables, custom dialogs, and theming while deliberately restricting myself to APIs present in that Qt version.

- I learned practical UDP and TCP programming with QUdpSocket and QTcpSocket, binding a socket to a chosen local adapter, running one receiver per connection so that datagrams on different endpoints are decoded only against the messages bound to them and never cross-contaminate, and handling both TCP client and server roles.
- I implemented serial (COM port) transmission through QSerialPort, setting baud rate and framing so the identical message model could be pushed over a serial link exactly as it is over the network.
- I built a byte-level command of binary protocols, coding big-endian layout with opt-in per-field little-endian that reverses only the numeric and float bytes at the final wire step, two's-complement encoding for signed and wide integers, fixed-width IEEE float and double, and the two-hex-digit XOR checksum that closes every NMEA-0183 sentence.
- I worked directly against the pcap and pcapng file formats, writing a PcapWriter that synthesises Ethernet, IPv4, and UDP or TCP frames and a reader path that re-parses them, and I demonstrated a round-trip in which payload, source and destination IP addresses, and ports are preserved byte-for-byte and the output opens cleanly in Wireshark.
- I set up a multi-target qmake build that yields two executables from one shared core, vendored the MIT-licensed QXlsx library into both, and wrote a headless test harness that exercises the real import, codec, encode-decode, and pcap paths end to end, with all 33 of its checks passing.

Professional Learning Outcomes

- I strengthened my problem decomposition by reducing an open-ended, feature-heavy brief to a single shared model and one inverse contract, which collapsed a broad requirement set into a small number of invariants I could reason about precisely.
- I practised evidence-driven debugging, closing the simulator and reader into a loop so that any wire-format defect could be localised to the individual offending byte instead of being chased through its symptoms.
- I built the habit of independent, source-led learning, reading the NMEA-0183 and pcapng format descriptions and the Qt and QXlsx references to close the gaps my coursework never reached.
- I improved how I document and organise code, keeping the shared core as the one authoritative copy, generating in-app HTML manuals with search and a table of contents, and making every user-facing failure report both its reason and its remedy in a single collected dialog.
- I kept version control disciplined with Git, landing small self-contained commits and holding the build reproducible so that a clean checkout compiles both applications with no manual intervention.

5.3 Limitations and Future Scope

Although the suite meets its core goals and both applications build and run dependably, finishing it made me candid about where it falls short. Some limits are deliberate scope decisions I would revisit with more time; others are natural extension points the current architecture already leaves room for. I record the principal constraints first, then the improvements I would pursue.

- The simulator's encoder is capped at eight bytes (64-bit, quint64) per field, whereas the reader already decodes fields of arbitrary width through exact base-256 to base-10 conversion, so for very wide fields the two ends are deliberately asymmetric.
- ICD import is limited to Microsoft Word .docx files, read through QZipReader, so an interface control document delivered as a spreadsheet or a PDF cannot yet be brought in directly.
- In the reader's live mode the capture status label is shared across every receiver, so an operator running several connections at once has no per-connection view of health.
- The in-app manuals have fallen behind the code and still document removed behaviour such as CSV export and the old live-mode transport row, so they need a full rewrite and re-screenshotting.
- Extend the simulator's encoder to fields wider than eight bytes so its output width matches the reader's unbounded decode, reusing the same exact base-256 arithmetic.
- Add an .xlsx-based ICD importer, which is cheap to build because QXlsx is already linked into both applications.
- Give the reader per-connection live status so each receiver reports its own packet counts and errors rather than sharing one label.
- Auto-detect the bytes-versus-words offset unit during ICD import, extending the existing offset-unit selector so the base is inferred instead of set by hand.
- Regenerate the in-app manuals and their screenshots to match the shipped feature set, including the packet inspector, pcapng export, and adapter-bound sending.

Looking back over the whole project, the Universal Data Suite gave me the uncommon chance to own a piece of software from the wire format upward, and the single most valuable lesson was that a small set of rigorous invariants, one shared model and one inverse contract, holds a genuinely intricate system together far better than a large body of loosely coupled code. Forcing the sender and receiver to be exact mirrors imposed a clarity that repaid itself at every later stage, from the headless tests to the multi-connection routing. I leave the project confident that I can carry these habits, precise binary reasoning, a preference for one source of truth, and honest reason-and-remedy error reporting, into more demanding engineering work.